Admissibility: What a Manipulation Benchmark Score Certifies

Anonymous submission

Abstract

A manipulation success rate can be made of *location* instead of *looking*. We make that precise. Define a task's *placement diameter* D as the largest distance between any two of the target positions the benchmark ships, and call a suite *lookup-admissible* at tolerance δ when a table indexed on the task index alone reproduces every shipped position to within δ. If $D \leq \delta$ and the embodiment cannot resolve δ, that table achieves whatever the controller achieves given the true position — and no function of episode outcomes can distinguish it from a policy that perceives.

Measuring all four LIBERO suites [6], exactly one is lookup-admissible: every LIBERO-Object task has $D \leq 2.34\,\text{cm}$, and no task in any other suite does (0 of 28; their mean diameters are 3.65–6.21 cm). LIBERO-Object's largest diameter is smaller than every other suite's mean. Six of its ten targets take just two `float64` values, one unit in the last place apart, across all fifty shipped states.

We then run the policy the proposition describes. It reads the task index, opens the benchmark's own shipped files, emits two constants, and consults the episode never — no camera, no language, no simulator, no learned parameter. It scores 6/10 against our released policy's 8/10, and an exact paired test returns $p = 0.50$: **the benchmark cannot separate a trained policy from one that has never seen the episode**, which is what the corollary says no score can do. The same machinery given uninformed goals scores 0/10, so the task is not trivial; the benchmark is.

The failures are the tell. In all ten trials the blind constant closed on the object to within 2 mm, and every failure ended at the step cap still holding it. What a constant cannot do here is the *basket* — the one sub-goal this suite randomises. **The diameter predicts where lookup suffices sub-goal by sub-goal inside a single task.**

Across seven of ten tasks that released policy scores 8 of 70, every success on the one object it was trained for.

Finally, certification. Probes alone cannot certify: two heads whose attribution profiles agree to 0.73 cm score 0.000 and 0.700 deployed. And a behavioural certificate is joint in *(head, stack)* down to a component no requirements file pins — holding weights, seeds and every package version fixed and changing only the renderer leaves the success *rate* identical and flips 40% of the individual episodes.

	probes from	cannot say
Benchmark degeneracy, outside robotics		
Torralba & Efros [11]	dataset identity	—
HANS [8], artifacts [4]	hypothesis only	where in a stack
ImageNet-v2 [10]	a second test set	—
Shortcuts [3]	general statement	—
Perturbation, in robotics		
LIBERO-PRO [12]	re-running with perturbed scenes	how much was gameable *a priori*
LIBERO-Plus [2]	seven axes, ten VLAs	which stage reads the shortcut
robomimic [7], COLOSSEUM [9]	re-collection, perturbation	what the shipped files already permit
This paper	**shipped files, before training**	—

Table 1: Every prior entry needs a trained model and a run to say anything. Placement diameter is computed from the benchmark's own initial-state files with no policy, no simulator and no training, which is what lets it bound what a score *could* certify rather than report what one model did.

1 The question

A policy scores 0.700 on a manipulation benchmark — 35 successes in 50 trials on one LIBERO-Object task; it is our own policy, and every number in this paper is traceable to a file we ship. What has been certified?

The intended reading is a capability claim: the policy perceives the named object, locates it, and manipulates it. The reading this paper is about is narrower and much cheaper: the policy has learned *where the object always is*. Both readings produce the same number. Distinguishing them is not a matter of looking harder at the policy — two policies can be indistinguishable to a probe and opposite in the world, and we exhibit such a pair — but of first measuring the benchmark.

The methodology is not new; the metric is. Table 1 places this work. Measuring a benchmark's degeneracy before crediting a model is standard outside robotics, and our *blind* arm is the manipulation instance of the hypothesis-only baseline; we claim no novelty for the

move. That VLAs ignore language is likewise not our finding.

What that literature lacks, and what manipulation makes possible, is a metric *computable from a benchmark's shipped files before any model is trained.* The prior here is not a class frequency but a physical coordinate, and a physical coordinate can be measured exactly.

What those results leave open is the part a practitioner needs. They show *that* scores collapse under perturbation. They do not say how much a given benchmark could have been gamed in principle, which stage of a stack is reading the shortcut, whether the shortcut can be removed, or what evidence would count as showing that it had been. This paper answers those four questions for one suite and one stack, in that order, and the first answer is the one that gives the others their meaning.

Contributions. Table 2 is the map; each row names the section that measures it and the artifact it is measured from.

	contribution	the number
1	Admissibility defined and measured (§2)	LIBERO-Object: 10/10 tasks admissible. Every other suite: 0/28
2	The proposition executed (§4)	shipped-files constant 6/10 vs. trained policy 8/10, $p = 0.50$
3	The prediction made *within* a task (§4)	pinned sub-goal 10/10, randomised sub-goal 6/10, same controller
4	Four-layer mechanism audit, two layers outside the model (§6)	a scripted expert's constant, and our own checkpoint-selection loop
5	Certification is joint in (head, stack) (§8)	probe-identical heads score 0.000/0.700; the renderer flips 40% of episodes
6	Falsification record (§10)	two registered predictions failed; one repair claim withdrawn

Table 2: What this paper claims, where it is measured, and the number that carries it. Rows 1–3 are the argument; rows 4–6 are what it took to trust them.

Scope, stated once and up front. All results are simulation. Every confirmatory cell is one task and one object unless stated. Cell sizes are given with every number and no cell is quoted without its interval. Our external validation attempt is reported as a *negative* (§9): we could not reproduce a public checkpoint's published performance on our build, and we do not report probe results on a harness whose baseline we cannot reproduce.

2 Admissibility

We want a property of a *benchmark*, computable before any model exists, that says how much of a score a lookup table could have taken. Two quantities give it: how far a target moves across the states a suite ships, and how that movement is arranged.

2.1 Definition

Fix a benchmark task t that ships a finite set S_t of initial states, and let $x_t(s) \in \mathbb{R}^3$ be the pose of the task's target object in state $s \in S_t$. Poses are compared at a physical tolerance δ, below which two placements are not distinguishable by the embodiment under test.

δ does two jobs and we keep them apart. As a *reporting* tolerance it is a modelling choice, so we sweep it from 0.5 mm to 5 cm rather than pick one (Appendix A). As an *admissibility* threshold it is a physical property of the embodiment, not a choice: $\delta = 2.5$ cm is the tolerance our controller demonstrably has, measured in §7.1, and it is the value used wherever this paper calls a task admissible.

Definition 1 (Placement diameter)**.** *The* placement diameter *of task t is the largest distance between any two shipped target positions,*

$$D(t) \;=\; \max_{s,s' \in S_t} \|x_t(s) - x_t(s')\|.$$

D carries no free parameter, is in centimetres, and is exactly the quantity the next proposition needs: if $D(t) \le \delta$ then *some* single constant is within δ of every shipped position. A second, finer question is how the variation inside that diameter is distributed — one tight cluster, two clusters, or a spread — and for that we use an entropy.

Definition 2 (Placement entropy)**.** *Partition $\{x_t(s) : s \in S_t\}$ into the connected components of the graph joining two placements whenever they lie within δ — equivalently, single-linkage agglomeration at radius δ. Let p_i be the fraction of states in component i. The* placement entropy *of task t is*

$$H_\delta(t) \;=\; -\sum_i p_i \log_2 p_i \quad \textit{bits},$$

and the placement entropy of a suite is the mean over its tasks.

We cluster rather than bin, and the difference is not cosmetic. Quantising onto an axis-aligned grid is not translation invariant: a tight cluster straddling a cell boundary splits in two, and a pinned task is then scored as diverse. Our own first version of this measurement had that defect — one LIBERO-Object task scored $H = 0$ at $\delta = 1$ cm and $H = 1$ at $\delta = 2$ cm, purely from where the origin fell. Definition 2 reads only pairwise distances, so no choice of origin can change it.

$H_\delta(t) = 0$ says the target starts in the same place every time, to within δ. The ceiling is $\log_2 |S_t|$, attained when every shipped state places the target differently.

Two further choices bound what the definition can be used for. We report the mean over tasks rather than pooling states across tasks, because pooling would credit a suite for *between*-task variation, and between-task variation is exactly what a task-indexed lookup gets for free. And H_δ is a *decreasing* function of δ under Definition 2 (coarsening can only merge components), so it is monotone and the whole curve is informative; we report it rather than a point, because δ is the one place an author could tune the answer.

Definition 3 (Order-k lookup-admissibility)**.** *A suite is* order-k lookup-admissible *if there is a map from the task index to a set of at most k position constants reproducing every shipped target position to within δ.*

The adjective attaches to the *suite* and the consequence to the *score*: a suite that is lookup-admissible renders its own scores inadmissible as evidence of grounding. We never use "admissible" as a virtue.

Assumption A1 (δ-robustness). The controller's success indicator is constant on δ-balls: for every $s \in S_t$ and every c with $\|x_t(s) - c\|_\infty \le \delta$, supplying c and supplying $x_t(s)$ yield the same episode outcome. A1 is an empirical property of the embodiment, not a definition, and §7.1 measures where it fails.

Proposition 1 (Lookup sufficiency)**.** *If $D(t) \le \delta$ for every task in a suite and A1 holds at δ, then the suite is order-1 lookup-admissible, and a policy that (i) reads only the task index, (ii) emits the corresponding constant, and (iii) is otherwise driven by an identity-independent controller, achieves on that suite whatever success rate the controller achieves given the true target position.*

Proof. Take c_t to be any shipped position of task t. By Definition 1, $\|x_t(s) - c_t\| \le D(t) \le \delta$ for every $s \in S_t$, so $t \mapsto c_t$ witnesses order-1 lookup-admissibility. The policy emits c_t on every state; A1 applies directly at each state, giving the same episode outcome as the true position. Averaging over S_t gives the claim. □

The hypothesis is $D \le \delta$ and not $H_\delta = 0$, and the distinction is not pedantic: $H_\delta = 0$ under Definition 2 says the placements are chain-connected at radius δ, which permits a cloud many times wider than δ, and A1 is a single-hop assumption that does not compose along such a chain. Four LIBERO-Object tasks are exactly this case ($H_{5\text{mm}} = 0$ with diameters of 1.19–2.34 cm), so the proposition is stated on the quantity that licenses it.

Corollary 1 (No score can separate them)**.** *On a suite with $D \le \delta$ throughout, no function of episode outcomes can distinguish the lookup policy of Proposition 1 from* any *policy supplying the controller a position within δ of the truth on every shipped state, however obtained — perceptually or otherwise — because under A1 the two induce the same distribution over episode outcomes.*

Corollary 1 is the sentence the rest of the paper exists to make concrete. It says the failure is not one of statistical power — no number of trials helps, because the two policies are not merely hard to tell apart but *identically distributed* in what the benchmark observes.

Proposition 1 is trivial as mathematics and is stated because its contrapositive is what benchmark scores are usually read as asserting. A suite with $D \le \delta$ cannot, by any score it reports, distinguish a policy that perceives from one that looks up — and the gap between them is exactly the capability the score is normally taken to certify. Note what the proposition does *not* say: it does not say the task is easy. The controller's own success rate under a correct pose is an empirical quantity, and if it is low the lookup policy scores low too. We measure both in §4.

2.2 Measurement

H_δ is computed from the benchmark's shipped files. No policy, no simulator and no training are involved, which is what makes it a property of the benchmark rather than of anyone's system.

LIBERO ships each task's initial states as a MuJoCo flattened state $[\,t \mid q \mid \dot{q}\,]$; every shipped $\dot{q}$ is zero, so the columns that vary across states are exactly the position degrees of freedom the suite randomises.

Recovering *which* object owns which column requires care. The fixed layout $[\,t \mid 9 \mid N \times 7 \mid \dot{q}\,]$ that works for LIBERO-Object predicts a width of $19 + 13N$ and fails for every suite containing an articulated fixture (LIBERO-Spatial ships width 92 against 5 declared objects). We therefore compile each task's model and read `jnt_qposadr` with the joint names, which maps every column onto a named body. The two passes agree where both apply.

Table 3 is the measurement.

The separation is exact, and has no free parameter. A single constant serves task t iff $D(t) \le \delta$. Taking $\delta = 2.5$ cm — a tolerance the controller demonstrably has, since §7.1 shows it still succeeding under ± 2 cm of deliberate displacement:

> **All ten LIBERO-Object tasks satisfy $D \le 2.5$ cm. No task in any other suite does** — 0 of 28.

LIBERO-Object is therefore order-1 lookup-admissible as a whole suite, and Corollary 1 applies to it without qualification. The margin is not narrow: LIBERO-Object's *largest* diameter (2.34 cm) is smaller than every other suite's *mean* (3.65–6.21 cm), and six of its ten tasks have

	diameter D (cm)		H_δ (bits)		tasks
suite	mean	max	0.5mm	5mm	$D \leq 2.5$cm
Object	**0.60**	**2.34**	2.11	0.00	**10/10**
Spatial	4.29	8.90	5.48	2.04	**0/10**
Goal	3.65	4.94	5.59	1.53	**0/8**
Long	6.21	6.57	5.63	4.27	**0/10**

Table 3: Every LIBERO suite, measured from its shipped initial states alone (50 per task) — no policy, no simulator, no training. The **diameter** D is the largest distance between any two shipped positions of a task's target; it is the quantity Proposition 1 needs, and it has no free parameter. The **entropy** H_δ says how the variation inside a diameter is arranged (ceiling $\log_2 50 = 5.644$ bits), and does have one. The last column counts tasks admissible at the $\delta = 2.5$ cm the controller demonstrably has (§7.1): **LIBERO-Object is admissible on all ten of its tasks and no other suite on any of its twenty-eight**, and LIBERO-Object's *largest* diameter is smaller than every other suite's *mean*. Denominators are *resolvable* tasks: two LIBERO-Goal tasks name a fixture with no free joint and so have no placement to measure; they are excluded and counted rather than scored as unpinned. Artifacts: `results/placement_diameter.json`, `results/placement_entropy.json`.

$D = 0$ to the precision `float64` expresses (two values, one ULP apart, $\sim 3 \times 10^{-17}$ m).

The entropy columns say how the variation inside a diameter is arranged, and add one fact the diameter cannot: at $\delta = 5$ mm the six pinned tasks and the four jittering ones *both* reach $H = 0$, because each of the four is a single chain-connected cloud rather than two separated modes. That is why the proposition is stated on D: chain-connectedness at 5 mm is compatible with a 2.34 cm cloud, and no constant is within 5 mm of that. We report both quantities and license the claim on the one that supports it.

How pinned is pinned. In 6 of Object's 10 tasks the target's start position takes exactly *two* distinct `float64` values, one unit in the last place apart ($\sim 3 \times 10^{-17}$ m) on a single axis — constant to the precision the format can express.

The remaining four have per-axis standard deviations of 0.25–0.58 cm but diameters of 1.19–2.34 cm — still inside the 2.5 cm tolerance, and a reminder that a standard deviation is not a bound.

Across tasks the structure is tighter still: the ten targets occupy two table positions 22.1 cm apart, five tasks each (Figure 1A), so a *two*-constant table covers the whole suite — order-2 lookup-admissibility, in the language of Definition 2.

This is not carelessness, and saying so sharpens the finding. LIBERO-Spatial's ten tasks all name a "black bowl" and are individuated *by* position; freezing placement would make the suite unsolvable. LIBERO-Object individuates by identity and therefore *can* freeze placement without ambiguity; Figure 1B shows what a suite that cannot looks like. The suite that does is precisely the suite whose scores are read as evidence of object grounding — which is where lookup-admissibility is most misleading.

One property generalises. The target's start *orientation* is bit-identical across all 50 states in 37 of the 38 resolvable tasks in all four suites. A policy may assume a canonical target rotation suite-wide and never be contradicted.

3 A glass-box stack

The artifact is an instrument, not a contribution. It is built to be traceable end to end, and its size (30.2 M deployed) is what makes an end-to-end trace feasible rather than a claim about efficiency.

component	params	trained	deployed
YOLO-World-S detector	13.0 M	never	frozen
world model (TRM)	9.97 M	stage A	frozen, inert†
trunk heads	7.0 M	stage A	frozen
goal heads + gates	0.24 M	stage B	frozen
deployed	**30.2 M**	17.2 M trained	all frozen

Table 4: Parameter ledger. There is *no* language model: the open-vocabulary detector's own text tower supplies every text embedding, so one frozen network is the only vision encoder *and* the only text encoder. †Inert in the one cell we have ablated: a persistence baseline reproduces the held-out cell exactly. We state it for that cell only.

Text is parsed into source/target phrases; the detector is prompted with them and returns a source box. A grasp head maps (uv, conf, proprio, box_emb, frame_emb) to a world grasp point — note the absence of any text argument, which is Figure 5's point and §8's architectural evidence. A place head maps the command embedding to a place (x, y), latched once per episode inside `reset()` and never re-read. A non-learned state machine converts goals into servo commands within a ± 6 cm probe envelope.

Three words, used throughout.

- **cell** — one evaluation: a fixed (head, flags, seed, n) run to completion, always quoted with its n and its interval.
- **band** — the set of shipped initial states a cell replays. The harness fixes it; it is not sampled.
- **burned** — a band we have looked at and let influence a choice. From that moment it no longer measures generalisation, and no amount of re-running un-burns it.

Protocol. LIBERO at commit `8f1084e3`, MuJoCo 2.3.7, robosuite 1.4.1, torch 2.8.0, ultralytics 8.4.115,

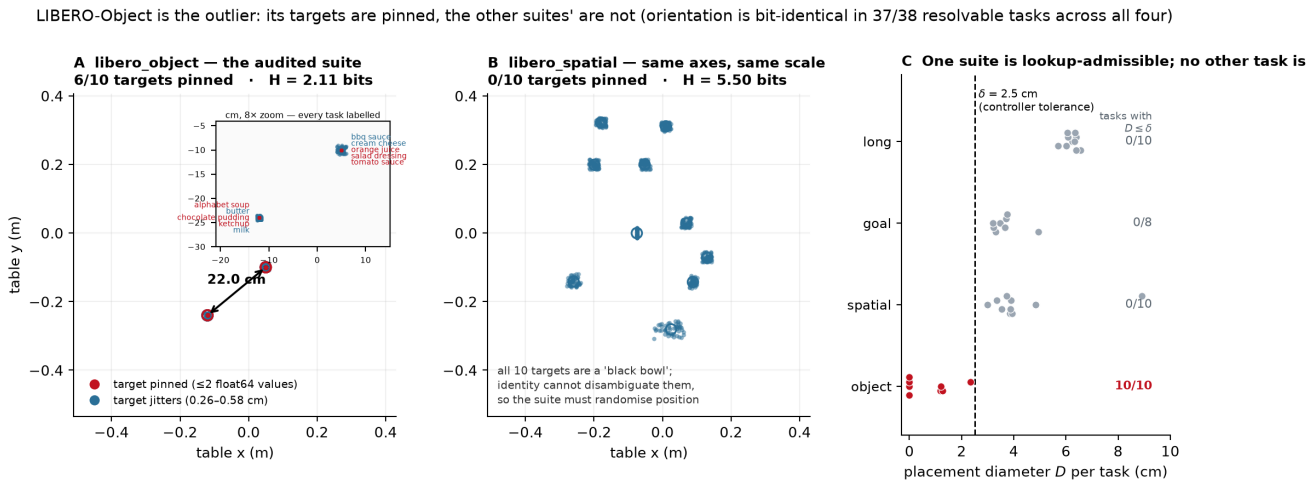


Figure 1: Placement forensics. **A** LIBERO-Object primary targets, 10 tasks × 50 shipped states; six collapse to a point and the ten task means occupy two clusters 22.1 cm apart. **B** LIBERO-Spatial on identical axes: every target scatters, and it must, because all ten of its targets are a "black bowl". **C** placement entropy per suite against the 5.644-bit ceiling. Generated by `paper/render_fig1_placement_forensics.py` from `results/suite_forensics_joints.json`; per-task rows are in that artifact and SHA-256 digests of every init-state array are in `results/suite_forensics_columns.json`.

numpy 2.2.6, Python 3.10, `MUJOCO_GL=osmesa` — §8 explains why that last entry is not boilerplate.

Which states compose a cell is derivable from its command line, not logged. The harness sets $\texttt{trial\_seed} = \texttt{seed} \cdot 1{,}000{,}003 + \texttt{trial}$ and indexes state $\texttt{trial\_seed} \bmod 50$; since $1{,}000{,}003 \equiv 3 \pmod{50}$, trial i at seed s replays state $(3s + i) \bmod 50$:

cell	seed, n	states
dev	0, 10	0–9
held-out	20, 10	10–19
held-out n=50	20, 50	**0–49**
untouched	40, 30	**20–49**

Table 5: Which of the 50 shipped initial states composes each evaluation cell, derived from the harness's seeding rule rather than logged. Because $\texttt{trial\_seed} = \texttt{seed} \cdot 1{,}000{,}003 + \texttt{trial}$ and $1{,}000{,}003 \equiv 3 \pmod{50}$, trial i at seed s replays state $(3s + i) \bmod 50$. The bolded rows are the ones that matter: an n=50 cell is the entire state set and therefore *contains* the dev and held-out bands rather than being disjoint from them.

The answer is not flattering: **the n=50 cells are not disjoint from the dev and held-out bands — they contain them.** Fifty trials over fifty shipped states is the whole set. The untouched band exists to supply one clean cell, and §7 reports it.

4 The constant that saturates the benchmark

Proposition 1 says that on a suite whose placement diameter falls below the embodiment's tolerance, a table indexed on the task index suffices. §2 showed LIBERO-Object is such a suite. This section executes the policy the proposition describes and reports what it scores.

The vehicle is our own stack (§3): a learned goal head that emits a world grasp point, driving a non-learned pick-and-place controller. That factorisation is what makes the experiment clean — we can replace the *goal supply* and change nothing else. Same controller, same trunk, same protocol, same seeds, same initial states; only where the goal comes from differs.

- **oracle** — the simulator's true target pose, re-read every tick. The controller's *ceiling*.
- **random** — a draw from the observed extent of the scene's own objects, re-drawn per episode. The controller's *floor*: what the machinery scores when the goal carries no information about where the object is.
- **reset-oracle** — the true target position read *once* at reset and held. On a task with $D = 0$ this is numerically identical to a per-task constant, but it still reads the simulator; it is **not** Proposition 1's policy, and the gap is stated below.
- **learned head** — our released policy, for reference.

The floor is zero. With an uninformed goal (Figure 2) the machinery scores 0/10 $[0.00, 0.28]$: eight discordant pairs, all favouring the learned head, exact McNemar $p = 0.0078$. **The task is not trivial.** Whatever the constant achieves, it does not achieve it because the controller would have succeeded anyway. This is the control that makes the rest of the table mean something.

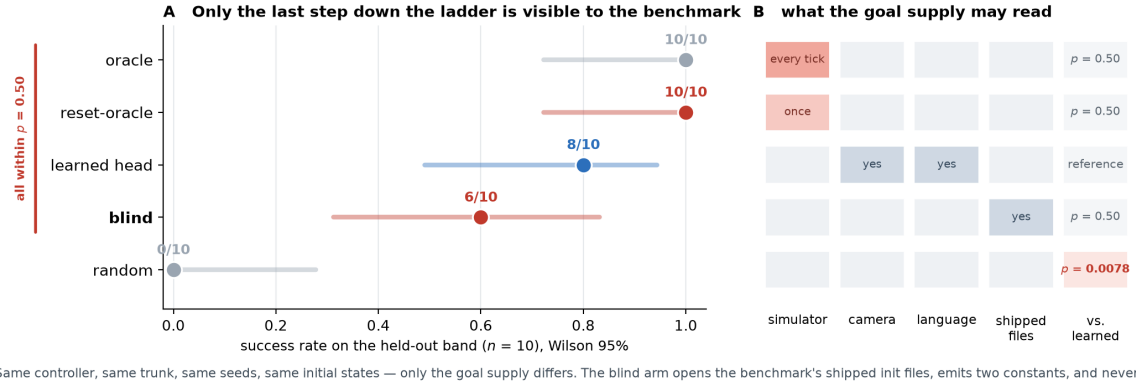


Figure 2: **The goal-supply ladder.** Five arms, one controller, one trunk, one protocol, one set of initial states; the only difference is where the goal comes from. **B** is the experiment: descending the ladder removes information, from the simulator every tick, to the simulator once, to a camera and a sentence, to nothing but the benchmark's own published files, to nothing. **A** is the result: the benchmark resolves only the final step. Every arm above `random` is within $p = 0.50$ of the trained policy under exact McNemar on matched trials, and `blind` — which never reads the episode — beats `random` at $p = 0.031$. Artifacts: `results/pod_cells.json`, `results/blind_cells.json`.

The strictly-blind policy, and what it cannot see. The reset-oracle still reads the simulator, twice: the target position once at reset, and the container position. Proposition 1's policy reads *neither*. The **blind** arm takes the task index, opens the benchmark's own shipped initial-state files, and emits two constants — the mean shipped position of the target and of the container — and then never consults the episode again. No simulator read, no camera, no detector, no text, no learned parameter anywhere in the goal supply.

On task 0 those constants are $(-0.1200, -0.2400, 0.0150)$ and $(0.0023, 0.2605)$: printed by the run, and reproducible from the shipped files without a simulator installed.

It scores 6/10 $[0.31, 0.83]$ on the same held-out band. Against the released head's 8/10 the two discordant pairs both favour the head and exact McNemar returns $p = 0.50$: **the benchmark cannot separate our trained policy from a policy that has never seen the episode.** Against the uninformed floor's 0/10 the six discordant pairs all favour blind, $p = 0.031$: the constants carry real information and are not riding a lucky controller.

Against the reset-oracle's 10/10, four discordant pairs all favour the oracle at $p = 0.125$. One privileged look at reset does buy something the shipped-file mean does not, and n=10 cannot resolve how much.

Where it fails is where the suite randomises. The per-trial telemetry says something sharper than the rate does. In **all ten** trials, failures included, the gripper closed on the target with a minimum end-effector-to-object distance of **0.002** m. The blind constant *never* missed the object. All four failures ran to the 300-step cap with the object still 8 mm from the gripper — grasped, never placed.

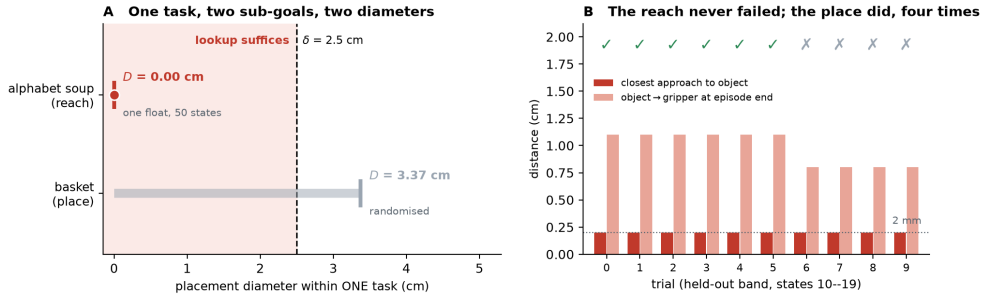
That is the admissibility argument making a *within-task* prediction and the measurement meeting it. This task carries two sub-goals with two diameters (Figure 3): the alphabet soup, whose fifty shipped positions are the *same float*, $D = 0.000$ cm; and the basket, which the suite genuinely randomises, $D = 3.37$ cm. A constant is sufficient for the first and insufficient for the second, and 6/10 is the composition of those two facts. The quantity we compute from the shipped files before running anything predicts, sub-goal by sub-goal inside a single task, where lookup suffices. Per-trial outcomes and the paired tests are in `results/blind_cells.json`.

The reset-oracle saturates the benchmark, and is statistically indistinguishable from the policy. The oracle scores 10/10, so the controller's envelope is not the binding constraint. The reset-oracle *also* scores 10/10, against the released head's 8/10 — and the paired test returns $p = 0.50$.

We do not report that p as a limitation of the cell. **It is Corollary 1 arriving on schedule.** On a suite whose target never moves, the score is not a low-powered instrument for separating lookup from perception; it is not an instrument for it at all. A larger n would not help. Under A1 the two policies are not merely hard to tell apart — they are identically distributed in everything the benchmark observes.

So the experiment's content is not "the constant won". It is that a benchmark scored a task-indexed constant and a trained policy the same, which is what Proposition 1 and Corollary 1 jointly predict.

What this licenses, and what it does not. It does not show our released head is a memoriser; §6 shows what it actually reads, and the answer is more interesting. It shows that **on this suite the score cannot distinguish**



The same constant-driven controller performed both sub-goals. It reached the pinned object in 10/10 trials and missed the randomised basket in 4 — the split the diameter predicts, computed from shipped files before anything ran.

Figure 3: **Admissibility predicts sub-goal by sub-goal, inside one task. A** "Pick up the alphabet soup and place it in the basket" has two sub-goals with two diameters: the soup occupies the *same* `float64` position in all fifty shipped states ($D = 0.00\,\text{cm}$), the basket does not ($D = 3.37\,\text{cm}$). **B** What the blind constant did, per trial. It closed on the object to within 2 mm in *all ten*, failures included; the four failures ran to the step cap with the object still 8 mm from the gripper — grasped, never placed. The same constant-driven controller performed both sub-goals, so the diameter is the only thing distinguishing them. Artifacts: `results/suite_forensics_joints.json`, `results/blind_logs/blind_t0_trials.txt`.

the two, because the cheapest possible policy saturates it. A benchmark on which a constant is at ceiling is not measuring grounding, whatever its leaderboard is titled. We report the head–oracle gap as *located* rather than measured: two discordant pairs at n=10 is $p = 0.50$, and we will not call that a difference.

One asymmetry, stated rather than buried. The anchors replace the *grasp* goal only; the place target is the true container in all arms. The random arm is therefore a *conservative* floor — an upper bound on how badly a truly uninformed goal supply would do — and we read it as such.

task	score	Wilson 95%
0 (alphabet soup, trained)	**8/10**	[0.49, 0.94]
1–6 (six other objects)	**0/10** each	[0.00, 0.28]
total, seven tasks	**8/70**	[0.06, 0.21]

Table 6: The released head across seven of LIBERO-Object's ten tasks, taken in index order rather than chosen by outcome, n=10 each. Every success falls on the single object the head was trained for. This is the number that puts the rest of the paper's cells in proportion: the artifact is a one-object solution that a suite-level score would have reported as a policy. Three tasks were still running when the compute window closed and are omitted; every cell shown reached its planned n.

5 What the released policy actually is

The previous section's cells are one task, and so is every confirmatory cell in this paper. Before we audit that policy it is worth stating plainly what it is, because the number that names it is not the one a suite-level score would report. Running the released head across seven of the suite's ten tasks — taken in index order, not chosen by outcome — gives that number:

Eight successes in seventy trials, and every one of them on the single object the head was trained for. Seven of the suite's ten tasks: the remaining three were still running when the compute window closed, and we report completed cells only — every cell here is at its planned n=10.

This is the admissibility argument turning on its author. A suite of ten tasks sharing two placements and one basket let a single-object solution be described as a policy with a suite-level score. **The number a reader should carry away from this paper is not** 0.700**; it is** 0.700 **on task 0, with** 0.000 **next to it six times.**

It also bounds in advance what the audit and repairs of §6–§7 can have bought. They made one object work and made the failures on the others legible — the drift case study (Table 7, last row) is one of those objects, lifted from 3/50 to 22/50 by DAgger on the policy's own viewpoints. They did not produce a general policy, and no cell in this paper should be read as claiming one. We put this section before the audit rather than after it so that no reader reaches the repairs holding a larger idea of the artifact than the artifact supports.

6 The audit: four layers

"The policy memorised the location" is not one hypothesis but four, at four stages — causal confusion [1] spread across a pipeline rather than confined to a network — two of which are not in the model at all — and those two are the ones a black-box perturbation study cannot reach (Figure 4).

Three of the four are described here; the fourth, being the one the instruments found rather than confirmed, gets its own subsection.

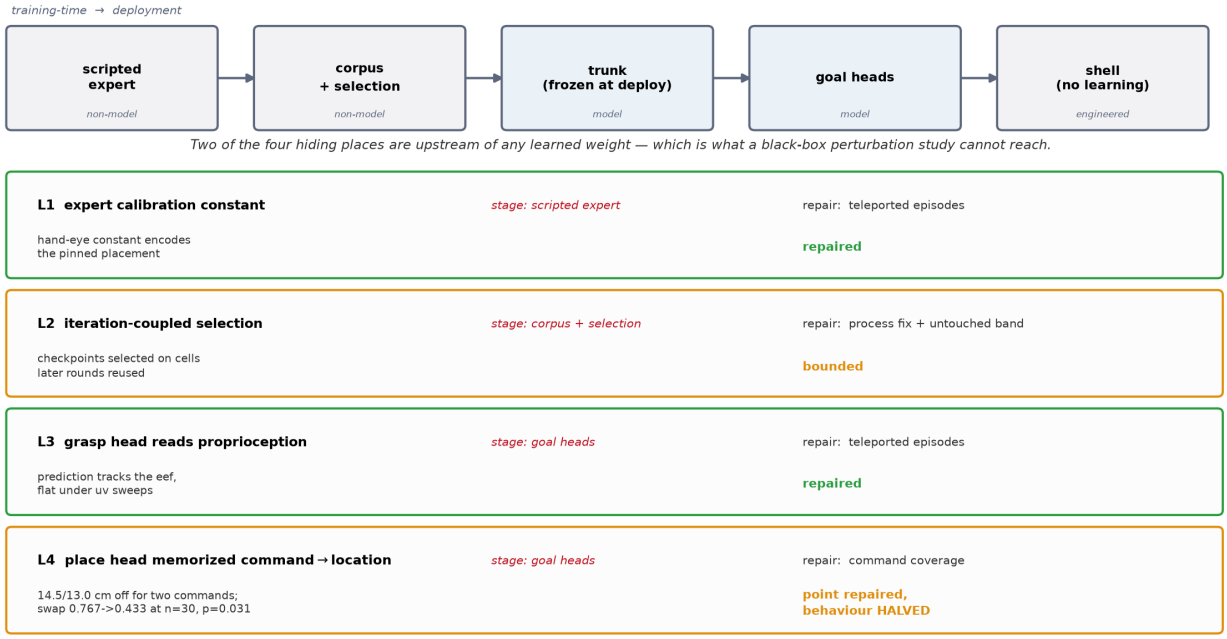


Figure 4: Where a fixed placement can hide in one stack, and the status of each site. Two of the four sit upstream of any learned weight.

Layer 1: the expert's calibration constant. The scripted demonstrator carried a hand-eye offset fitted once, on a scene whose target never moved. That constant therefore *encoded* the pinned placement, and every demonstration inherited it — so the corpus taught the pose before any network saw a pixel. Detected by reading the expert, not the policy. Repaired by re-baking ten episodes with the source object displaced, which breaks the constant's validity and forces the demonstrator to look; the repaired corpus lifts the dev cell from 0/10 to 7/10 on the same episodes.

Layer 2: the selection loop. Our own procedure, not the artifact's: at each round we chose a checkpoint using cells that later rounds reused, so selection pressure accumulated on one band of initial states. This is invisible to any measurement of the model and is the reason §7 runs an untouched band at all. It is *bounded* rather than repaired — one cannot un-look at data — and the bound is that thirty never-inspected states score within noise of the burned ones.

Layer 3: the grasp head's proprioceptive shortcut. Substitution attribution showed the predicted grasp point tracking the end-effector and barely moving under *uv* substitution: the head had learned to predict where the object is from where the arm is, which on a pinned suite is a sufficient statistic. The same teleported episodes repair it, and the released head's profile inverts — it now moves further on its visual channels (3.43/6.57 cm) than on proprioception (1.93 cm).

6.1 Layer 4: where the language goes

Swapping the instruction while holding the scene and the success predicate fixed collapses the released head from 8/10 to **0/20** $[0.00, 0.16]$; on the ten trials that pair with the reference cell, eight discordant pairs all one way, exact McNemar $p = 0.0078$.

Telemetry places the failure *downstream of approach*: the policy still reaches the true object as closely as baseline while the grip-close rate falls from ∼0.67 to ∼0.26. Driving the detection prompts and the place-head embedding from *different* instructions isolates the collapse to one channel, with no interaction.

Combined with the architecture: **object selection, approach and grasping run with no language input at all, and the entire language channel is a single cached** (x, y). Ask this policy for the butter and it finds, approaches and grasps the alphabet soup at baseline rate; it fails only when that one coordinate is wrong.

7 Repairs, and one we withdraw

Three of the four layers admit a repair, and we report what each bought. One claim from the previous version does not survive being run at power, and we retract it here rather than leave it standing.

The untouched band. Both n=50 cells contain the burned states, so we ran the released head on states 20–49, which no selection look ever reached, with the prediction recorded before the run ($[0.50, 0.75]$, falsifiers named in both directions). Result: **20/30 = 0.667** $[0.49, 0.81]$,

layer	where	evidence	repair	verification / status
1. expert calibration constant	scripted expert (non-model)	hand-eye constant encodes the pinned placement	placement teleportation	0/10 → 7/10 dev, paired. **Repaired**
2. selection loop	our own procedure (non-model)	checkpoints chosen on cells later rounds reused	process fix; bands frozen	untouched band 20/30 vs 0.700: no detected inflation. **Bounded, not removed**
3. grasp-head proprio.	`GraspPointHead`	prediction tracks the eef, flat under uv sweeps	same teleported episodes	attribution flips to visual. **Repaired**
4. place-head cmd→location	`PlaceHead`	correct basket for the trained command, 14.5/13.0 cm off for the other two; swap 8/10 → **0/20**, p=0.0078	command coverage	place *point* 14.15 → 0.78 cm **repaired**; swap *behaviour* 0.767 → 0.433, p=0.031 — **halved, not fixed**
(separate) appearance drift	deployment viewpoints	NN-cosine gap on the policy's own trajectory	DAgger on own viewpoints	3/50 → 22/50 vs 3/50 control, $p < 10^{-4}$. **Repaired**

Table 7: Master audit table. Layers 1–4 are the placement-memorisation layers; the last row is a drift case study, *not* one of the four; it appears here only because its repair is easily mistaken for one of theirs. Two rows are deliberately not marked repaired.

against the published 0.700 (35/50) a difference of −0.033, Newcombe $[-0.24, +0.16]$. That interval assumes independent samples and the samples are *not* independent: the n=50 cell spans states 0–49 and so contains all thirty untouched states.

The interval is therefore conservative in the wrong direction, and we use it only to bound the effect as small. The clean comparison — untouched-30 against the twenty genuinely burned states — is the one we would run next.

We claim exactly this much: not that the burn is zero, but that it is small enough to hide inside ±0.20, which is what n=30 buys.

A repair we withdraw. Command coverage was reported as repairing the instruction-swap behaviour on the strength of 3/10 against a 4/10 baseline at $p = 1.0$. That is an equivalence claim from an underpowered cell, on a band where the head was already mostly failing — and a swap cannot visibly break a cell that is already broken. Re-run on a band where the coverage head scores well, and taken to n=30:

head	baseline	swapped	paired McNemar
released head	8/10	**0/20**	$p = 0.0078$
coverage	23/30	**13/30**	$p = 0.031$

Table 8: Substituting a different task's instruction, on a band where each head scores well and at a power the original cell did not have. Both heads lose accuracy, so command coverage does not eliminate the instruction dependence it was built to remove; it halves it. Paired McNemar over matched trial indices (same seed, same initial states, same renderer — only the instruction differs), exact two-sided.

At n=10 this contrast was $p = 0.125$ and we would have called it unresolved; at n=30 it is $p = 0.031$. **The coverage head collapses too.** A swap cannot visibly break a cell that is already mostly broken, which is why the original 4/10 baseline hid the effect.

What replaces the withdrawn claim is more useful than it was. Coverage does not eliminate the failure, it roughly *halves* it: the released head loses its entire cell (0.800 → 0.000, every discordant pair one way), the coverage head about a third (0.767 → 0.433). And the repair to the place *point* is not in doubt — 14.15 → 0.78 cm (Figure 6), measured directly rather than inferred from behaviour.

So: **command coverage fixes where the head aims and does not fully fix what it does.** A residual command→location dependence survives training on all three commands — a sharper statement of layer 4 than we had, and one no cell we ran before could have detected.

7.1 Randomisation is a curve, not a point

Our ±4 cm radius was chosen to sit inside the controller's ±6 cm probe envelope — a perturbation calibrated to the system under test. Sweeping it on the same draws separates two things that single number confounded:

Read conservatively, because n=10 supports no more: clearly worse at ±6–±8 than at ±2, but every interval is ∼0.5 wide and the undisplaced cell sits *below* ±2 cm, which cannot be a real benefit of displacement. That non-monotonicity is the honest measure of what this n resolves. **No single radius should be quoted as "passes randomisation"**, and a benchmark asking for one is asking for a number that does not exist.

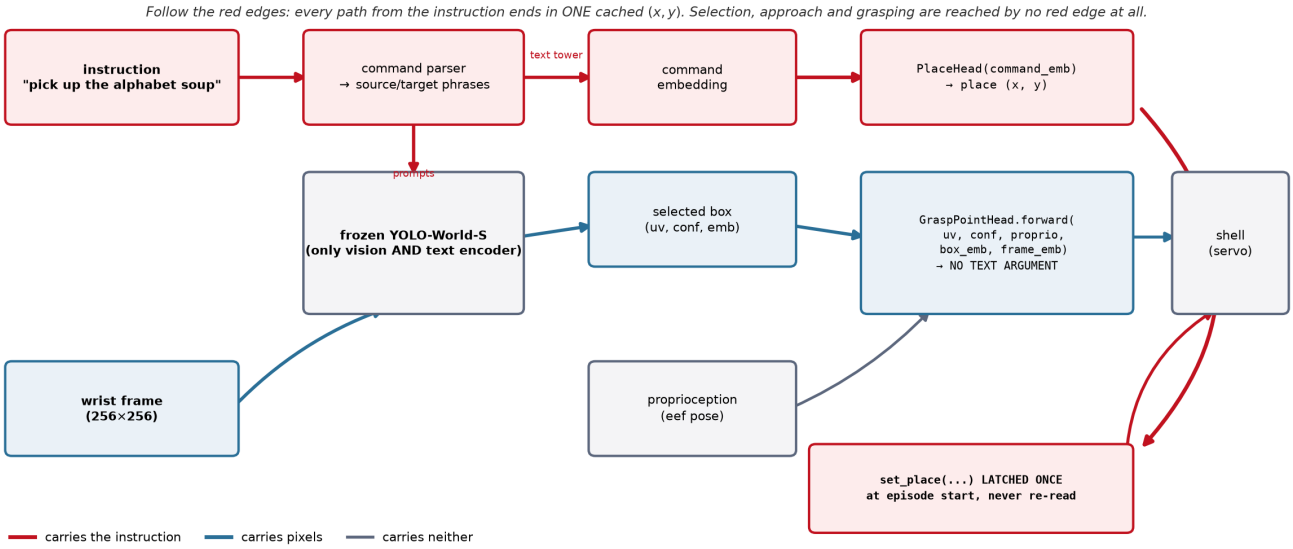


Figure 5: Every edge coloured by what it carries. The architectural half of the argument is a code fact: `GraspPointHead.forward` takes no text argument, so no red edge reaches object selection, approach or grasping, and every red edge terminates in one (x, y) latched once per episode and never re-read.

displacement	released head	Wilson 95%
none	4/10	[0.17, 0.69]
±2 cm	7/10	[0.40, 0.89]
±4 cm	4/10	[0.17, 0.69]
±6 cm (envelope edge)	2/10	[0.06, 0.51]
±8 cm (outside)	3/10	[0.11, 0.60]

Table 9: Sweeping the placement-randomisation radius on the same draws. Randomisation robustness is a curve, and at n=10 this curve is not even monotone — the undisplaced cell sits *below* ±2 cm, which cannot be a real benefit of displacement. Reporting any single radius as "passes randomisation" would quote a number this sample does not resolve.

head	uv	proprio	box_emb	frame_emb
v2	0.75	2.85	6.04	6.91
v2.1	0.71	2.13	6.26	7.58
v5 (released)	0.97	1.93	3.43	6.57

Table 10: Substitution-probe attribution, in cm of movement of the predicted grasp *offset*, per input channel. The counterexample is the first two rows: v2 and v2.1 agree to within 0.73 cm on every channel and score 0.000 and 0.700 when deployed. A probe evaluated on teacher trajectories cannot, on its own, certify off-manifold behaviour.

8 Certification

If a benchmark score cannot certify grounding, what can? This section reports three results about the certification problem itself, two of them negative.

8.1 Probes alone cannot certify

A substitution probe asks which input channels a head reads: swap one channel between two recorded ticks and measure how far the predicted grasp offset moves. It is annotation-free, cheap, and on-manifold — and that last property is fatal on its own.

The offset, not the absolute point: the head emits $\text{eef}_{xy} + \Delta$, so substituting proprioception moves the absolute prediction by the entire arm displacement for *any* head, and profiling that quantity reports ∼24 cm of "proprioception attribution" even for a repaired one. We made exactly this error when regenerating the measurement, and record it because it is the difference between a probe that answers "does this channel change where the head thinks the object is" and one that measures the parameterisation.

v2 and v2.1 differ by at most 0.73 **cm on any channel and score** 0.000 **and** 0.700 **deployed** (Figure 7). One caveat we state rather than let a reader discover: the substitution is between a *single* pair of recorded ticks — the two most separated in *uv* out of 329 — with no resampling over pairs and therefore no interval on the 0.73 cm. It is an existence proof that two heads can be probe-identical and behaviourally opposite, not an estimate of how often that happens.

A probe evaluated on teacher trajectories is an on-manifold instrument and cannot, alone, certify off-manifold behaviour. Probe evidence must be paired with behavioural randomisation.

The place head learned command → location, not basket. All ten tasks share one basket, so this was invisible until the instruction changed.

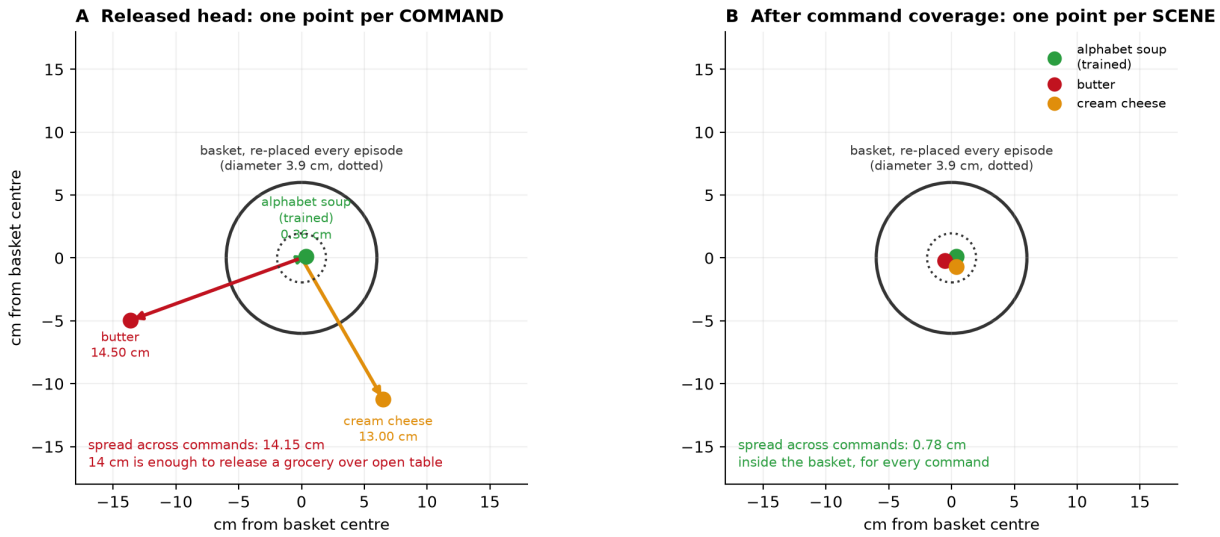


Figure 6: Layer 4 drawn rather than argued. The released head asked for three objects emits three place points, 14.5 and 13.0 cm from the basket. The basket itself is *not* fixed — it is re-placed every episode, with a per-task diameter of 3.66 cm (max 3.96) — so a memorised place point is wrong by 14 cm against a target that only ever moves by 4, and 14 cm is enough to release a grocery over open table. Radial distances are measured; angles are for legibility.

8.2 An instrument only ever shown firing is not yet an instrument

Our cross-instruction detection-agreement probe asks whether two objects' prompt chains select the *same box from the same frame*; a high same-box rate says the grounding stage carries no object identity. Every published number from it was a firing. Running it against every other object in the scene splits cleanly:

counterpart	n	same-box	median dist.
cream cheese, butter, pudding	6	**0.00**	0.817–0.818
tomato sauce	4	1.00	0.00069
bbq sauce, salad dressing	5	0.80	0.0043
ketchup	5	0.60	0.0048

Table 11: The positive control our detection-agreement instrument had never been given. "Same-box" is the fraction of frames on which two objects' prompt chains select the identical box; a high rate means the grounding stage carries no object identity. Run against *every* counterpart rather than only the ones that fire, the instrument separates cleanly — and the split is interpretable: the chains that collide are the cans and bottles, the ones that separate are the boxes. Deployed binding is blind *within* a shape class, which is the regime LIBERO-Object's groceries occupy. Small n (only 4–6 of 60 frames have both chains detecting) makes these demonstrations of discrimination, not calibrated rates.

The instrument registers difference when difference exists. The distances are bimodal — collisions at ∼0.004, separations at ∼0.82, nothing between — and a sweep over $\tau \in \{0.005, 0.01, 0.02, 0.05\}$ leaves every verdict unchanged. The split is interpretable and sharpens the claim: the chains that collide are the cans and bottles, the ones that separate are the boxes. **Deployed binding is blind *within a shape class***, which is precisely the regime LIBERO-Object's groceries occupy.

Two limits. Only 4–6 of 60 frames had both chains detecting, so these are small-n demonstrations of discrimination, not calibrated rates. And the contrast we originally designated the positive control — the basket — is not evaluable at all: the deployed source-area filter rejects basket-sized boxes by construction, so it compares zero frames. Our first version of the script read same_rate = 0 off that empty comparison and printed "instrument discriminates". A verdict now requires $n \geq 3$.

8.3 Certificates are joint in (head, stack), down to the renderer

We rebuilt the deployment stack on fresh hardware, pinning every version in Table 4's caption and verifying both load-bearing checkpoints by digest. The reference cell reproduces: 8/10 [0.49, 0.94] against a recorded 7/10, with 0.700 inside the interval.

Reproducing the *rate* is not reproducing the *experiment*. Holding seed, trial, weights and every package version fixed and changing only the OpenGL backend MuJoCo renders through:

The aggregate is identical and 40% of the episodes are not. Thread count, by contrast, changes nothing: three `osmesa` runs at 1, 4 and 128 threads agree on all eight logged fields to the printed precision while the wall clock falls from 220 s to 32 s (`results/thread_determinism.json`). Software and

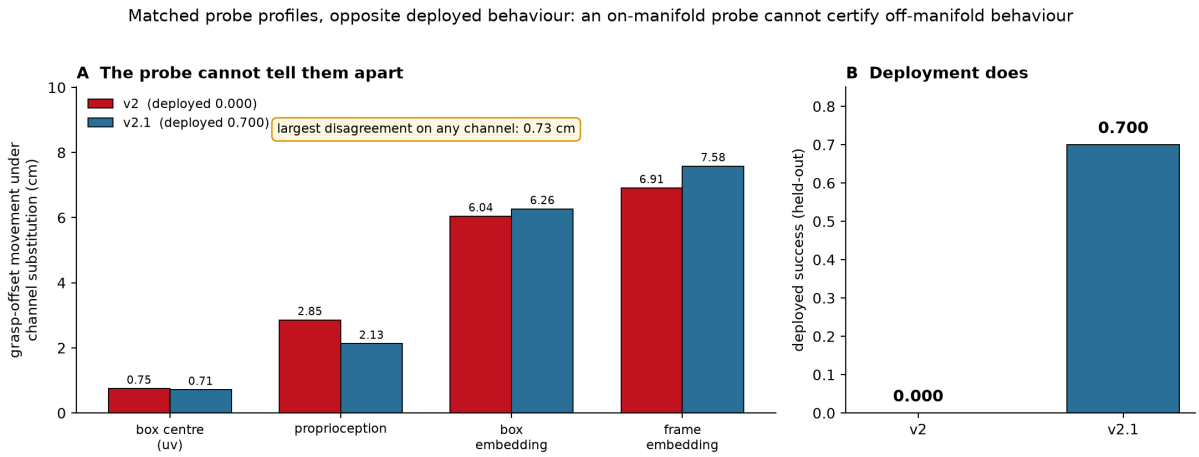


Figure 7: Two heads trained the same way on the same 10-episode corpus. Their attribution profiles agree to within 0.73 cm on every channel; deployed, they score 0.000 and 0.700. Regenerated by `scripts/attribution_profiles.py`.

	`osmesa`	`egl`
success rate	8/10	8/10
per-trial outcomes	**4 of 10 disagree** (2 each way)	

Table 12: Changing only the OpenGL backend MuJoCo renders through — same seed, same trials, same weights, same package versions. The aggregate is identical and 40% of the individual episodes are not. Matching a published rate is therefore weak evidence of having reproduced an experiment, and any paired statistic computed across a renderer change is invalid, because pairing assumes the trials are the same trials.

GPU rasterisation do not produce identical pixels, the detector is not invariant to that difference, and in this stack the detector is the only encoder.

Two consequences. Matching a published rate is weak evidence of having reproduced an experiment: the marginal can be right while the sample is different. And *any paired statistic computed across a renderer change is invalid*, because pairing assumes the trials are the same trials — every McNemar in this paper is within one backend, which we can now say we checked rather than assumed. `MUJOCO_GL` belongs in any reported stack; it was not in ours, or in any LIBERO evaluation we have read.

9 Transfer: a negative result

The instruments above are offered as portable. We therefore attempted to run them on a policy we did not build: the public `openvla-7b-finetuned-libero-object` checkpoint [5], evaluated through the same harness, the same trial→state map and the same success predicate.

We report a negative, and the discipline that produced it matters more than the outcome. The baseline returned 0/1 on a checkpoint published near 0.9. That is a harness defect, not a finding about OpenVLA, and we treated it as one.

Four defects were found and fixed: image orientation, the gripper convention (their model emits $[0, 1]$ with 1=close, robosuite wants $[-1, +1]$ with -1=close — without the conversion the jaw never closes and the policy *cannot* grasp), success extraction (the environment's `done` flag also fires on horizon), and a missing signal shield. We then swept four image orientations $\times$ the 0.9 centre crop their fine-tuned evaluation applies.

The checkpoint still does not approach the target in any configuration we tried; the end-effector never closes to within a grasp of it. **We therefore report no OpenVLA success numbers, and no distance statistics either** — the diagnostic runs that produced them were exploratory, were not written to a versioned artifact, and we decline to quote figures we cannot ship. An unvalidated baseline would make our instruments look powerful for entirely the wrong reason — every one of the four defects biased in that same direction — and that is the easiest way to fabricate an external-validation result without intending to.

We cannot separate the two live hypotheses: a remaining defect in our harness, or a checkpoint that does not transfer to a differently-built LIBERO. The second is exactly what §8 predicts and we decline to claim it, because the first is not excluded. The harness ships (`eval/openvla_eval.py`) so the question is answerable by someone with the matching build. Until then, the portability of our instruments is **unestablished**, and we make no claim that these instruments generalise beyond the stack they were built on.

10 Negative results and registries

A paper that only reports what worked cannot be checked. This section is the ledger: what we predicted before running, what failed, which instruments we retired, and the three ways our own results fooled us.

Pre-registration. Fourteen predictions are on record with their outcomes. Five earlier ones, all falsified. Two from the confound test (§10.1). Seven registered for this revision, of which five held — the untouched band, the oracle ceiling, the uninformed floor, the constant, and the released head's swap — and two failed: that the coverage head's swap cell would hold (it does not, p=0.031) and that the released head would be at floor once displacement left the controller's envelope (3/10, not ≈ 0).

Retired instruments. Quantifying deployed role binding was attempted six times and abandoned six times, each at a calibration gate written down before the instrument was trusted: a projected-origin ground truth (rejected — the object that succeeds 35/50 scored 0.111), its sign-corrected successor (rejected — the in-frame filter deletes the close-up target by construction), three text-tower discrimination probes (rejected — each failed to detect the working object's own phrase), and a box-spread metric (rejected — confounded by camera motion, and inverted). Deployed binding accuracy is reported as **unmeasured**.

10.1 A confound we had missed

Our own analysis of a blocked object was confounded: because the suite's two placement clusters align with which objects we happened to train on, "blocked" and "22 cm from the training position" were perfectly correlated in our design. Tested directly over all ten objects, position predicts the effect weakly (Spearman +0.19, p=0.60) and *teleporting each object 22 cm moves it in the wrong direction* (exact sign test p=1.0), while detector groundability predicts it (-0.71, p=0.027). The mechanism is appearance, not position — position is a nuisance term with mean $|\Delta| = 0.0066$, which is not zero either.

This does not contradict §2, and the distinction is worth stating. Those sections answer different questions. Placement is what the suite fails to randomise and therefore what a policy *may exploit*: that is a property of the benchmark, established from its shipped files, and it is what layers 1 and 3 were caught doing. Appearance is what limits *transfer to a new object* once that shortcut is repaired: a property of the frozen detector, and why the released head works on one object and not on six (§5).

A benchmark can be lookup-admissible and a policy can still fail for an unrelated reason. Both are true here, neither weakens the other, and what *would* have been a contradiction — position explaining both — is what the confound test ruled out.

We also report the caveats on this cell rather than only its conclusion. Both nulls are n=10, which is the size we decline to accept elsewhere (§7); the groundability predictor and the outcome are two statistics of the same detector forward pass, so their correlation is closer to a self-consistency check than to an independent prediction; and $p = 0.027$ across three tests does not survive a Bonferroni correction. The confound is *disconfirmed as a sufficient explanation*, which is what the design can support, and not more.

10.2 Three ways a result fooled us

Recorded because the failure modes generalise and *none* was caught by a conventional check — in all three the instrument was correct and the defect was elsewhere (Table 13).

what we saw	what it was	the general lesson
a perfect null: teleport gaps identical to control for all ten objects, to four decimals	a cached observation — the renderer returned the *pre*-teleport frame	implausible cleanliness is a symptom. The script now compares frames across the intervention and *raises* rather than emitting
a defect count withdrawn as unsupported	the appendix we could not find existed	"I could not find it" is a claim about a search, not about a corpus
7/7 against a published 7/10; we began writing up a failure to reproduce	a censored sample. The complete cell is **8/10**	successes end episodes early and failures run to the horizon, so any interim harvest is biased toward successes. **Verifying the instrument does not verify the sampling**

Table 13: Three self-inflicted errors and their repairs. The third is the one we would most expect others to hit: every check we ran had passed, and the defect was in *when* we looked. The harvester now reports each cell against its planned n and refuses to report an incomplete one.

11 Discussion

What a score certifies. On a suite whose placement diameter sits below the embodiment's tolerance, a success rate certifies that the policy can execute a manipulation *given* a correct target pose — and nothing whatever about how it obtained one. That is not a rhetorical claim: on LIBERO-Object a constant achieves the ceiling and an uninformed goal achieves zero, so the score's entire dynamic range sits between "has the pose" and "does not", with grounding never tested. Placement entropy makes this checkable before any policy is trained.

Shortcuts live in pipelines, not only in models. Two of our four layers are outside the network: a scripted expert's calibration constant and our own checkpoint-selection loop. No amount of perturbing a trained model from the outside would have found either. This is an argument for glass-box artifacts in benchmark-validity work, and it is the main reason our stack is small.

Certification is a system property. Probes cannot certify alone (two heads, 0.73 cm apart, 0.000 vs 0.700). Behaviour must be paired with randomisation. And the certificate is joint in *(head, stack)* down to a component no requirements file pins: the same weights on the same seeds give the same rate and a different 40% of episodes under a different renderer.

Limitations. One simulator, one benchmark suite, one robot; wrist camera only; confirmatory cells are one task and one object; the held-out band is partially burned and the disclosure is in §3; the on-manifold limit of substitution probes is a limit on our own instruments; and transfer to an external policy is **unestablished** (§9). Placement entropy is defined for suites that ship enumerable initial states and would need restating for procedurally generated ones.

11.1 Recommendation to benchmark authors

Both quantities are cheap to compute and cheap to report. Table 14 is what we would ask of a suite.

practice	cost	why
publish D and H_δ per task	one script, no training	tells a reader what a score can certify
set a floor on D	re-sample the init files	a suite individuated by identity must randomise placement, or identity is never load-bearing
ship a tier of radii	k extra eval configs	where a policy degrades depends on displacement relative to the controller's envelope (Table 9); one radius is a calibration, not a result

Table 14: Three practices, in increasing order of what they cost a benchmark's authors. LIBERO-Spatial already satisfies the second by necessity — all ten of its targets are the same bowl — and LIBERO-Object could satisfy it by choice.

12 Conclusion

A manipulation benchmark score is not a measurement of capability until the benchmark's admissibility has been measured. We gave the measurement, showed that exactly one LIBERO suite fails it, and ran the policy its failure predicts — which the benchmark cannot tell apart from the one we trained.

What survives is not the artifact but the procedure: measure the benchmark first, localise the stage second, repair with data third, certify jointly in *(head, stack)* — and report, as we have, the repairs that did not survive being powered up.

A Sensitivity of H_δ to the tolerance

The quantisation δ is a modelling choice, so the finding must not depend on it. Recomputing placement entropy across four tolerances spanning an order of magnitude:

suite	0.5mm	2mm	5mm	1cm	5cm
Object	**2.11**	**0.48**	**0.00**	**0.00**	0.00
Spatial	5.48	4.49	2.04	0.25	0.00
Goal	5.59	5.08	1.53	0.02	0.00
Long	5.63	5.40	4.27	0.37	0.00

Table 15: Mean placement entropy H_δ in bits, swept over an order of magnitude in the tolerance δ. LIBERO-Object is lowest at every tolerance, so the ordering does not depend on the modelling choice — but every suite reaches zero by $\delta = 5$ cm, which is a statement about coarse resolution rather than about LIBERO. That degeneracy is why the paper's claim is licensed on the diameter D (Table 3), which has no such parameter. Artifact: `results/placement_entropy.json`.

The ordering is unchanged wherever the measure discriminates at all. Coarsening δ lowers every suite's entropy, as it must — coarser radii merge components — and by $\delta = 5$ cm every suite reaches zero, which is a statement about coarse resolution rather than about LIBERO. This is precisely why the paper's claim is licensed on the diameter D (Table 3), which has no such parameter: at $\delta = 5$ cm every suite would look admissible on H, while on D only LIBERO-Object is, at any tolerance the controller actually has. Artifact: `results/placement_entropy.json`.

References

[1] Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *NeurIPS*, 2019.

[2] Senyu Fei et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.

[3] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.

[4] Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of NAACL-HLT*, 2018.

[5] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[6] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023.

[7] Ajay Mandlekar et al. What matters in learning from offline human demonstrations for robot manipulation. In *CoRL*, 2021.

[8] R. Thomas McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2019.

[9] Wilbert Pumacay, Ishika Singh, Jiafei Duan, Ranjay Krishna, Jesse Thomason, and Dieter Fox. THE COLOSSEUM: A benchmark for evaluating generalization for robotic manipulation. In *Robotics: Science and Systems (RSS)*, 2024.

[10] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do imagenet classifiers generalize to imagenet? In *International Conference on Machine Learning (ICML)*, 2019.

[11] Antonio Torralba and Alexei A. Efros. Unbiased look at dataset bias. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011.

[12] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.